

Capítulo 6

Diferenciación e integración numérica

En el capítulo 5 se mencionó las ventajas de usar polinomios para aproximar una serie de datos o pasar exactamente por ellos. Si esos datos provienen de una función, el polinomio en cuestión aproxima a dicha función en un intervalo en el cual se encuentran esos datos. También se destacó que la elección de los polinomios tenía la ventaja de que son fácilmente diferenciables e integrables, por lo cual es natural suponer que este tipo de aproximaciones nos sirvan para estimar de manera precisa la derivada de una función o su integral definida en un intervalo.

6.1. Diferenciación numérica

Si se toma en cuenta la definición de derivada de una función f en un punto x_0 de su dominio:

$$f'(x_0) = \lim_{h \rightarrow 0} \frac{f(x_0 + h) - f(x_0)}{h}$$

es natural proponer como aproximación

$$f'(x_0) \approx \frac{f(x_0 + h) - f(x_0)}{h}$$

para valores pequeños de h . Sin embargo, tomar un h muy chico en el denominador, nos genera un aumento en el error de redondeo. En esta sección se estudiarán ésta y otras aproximaciones a la derivada en un punto, y sus alcances.

6.1.1. Fórmulas de 2 puntos

Si se considera el desarrollo del polinomio de Taylor de la función f en el punto x_0 , truncando la serie en el término lineal, se obtiene:

$$f(x) = f(x_0) + (x - x_0) f'(x_0) + \frac{(x - x_0)^2}{2} f''(\xi_1)$$

para algún ξ entre x_0 y x . Evaluando en $x = x_0 + h$ para algún $h > 0$:

$$f(x_0 + h) = f(x_0) + h f'(x_0) + \frac{h^2}{2} f''(\xi_1) \quad (6.1)$$

con $\xi_1 \in [x_0, x_0 + h]$ despejando la derivada:

$$f'(x_0) = \frac{f(x_0 + h) - f(x_0)}{h} - \frac{h}{2} f''(\xi_1) \quad (6.2)$$

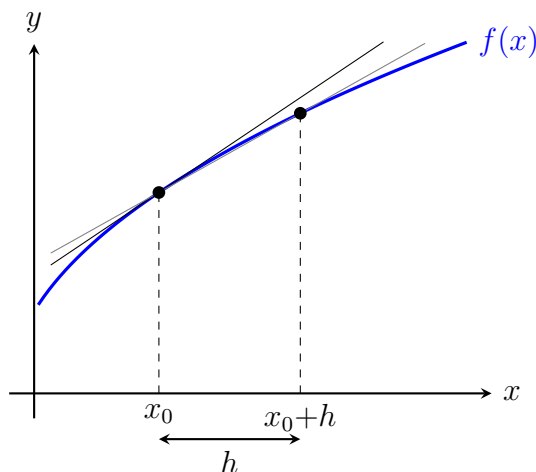


Figura 6.1: Diferencias finitas adelantada de dos puntos.

Si se evalúa ahora el polinomio de Taylor en $x_0 - h$, se obtiene:

$$f(x_0 - h) = f(x_0) - h f'(x_0) + \frac{h^2}{2} f''(\xi_2) \quad (6.3)$$

y al despejar la derivada

$$f'(x_0) = \frac{f(x_0) - f(x_0 - h)}{h} + \frac{h}{2} f''(\xi_2) \quad (6.4)$$

para algún $\xi_2 \in [x_0 - h, x_0]$.

Las ecuaciones (6.2) y (6.4), denominadas *fórmulas en diferencias finitas* (hacia adelante y hacia atrás respectivamente), nos proporcionan una aproximación a la derivada $f'(x_0)$ evaluando la función en dos puntos, y cuyo error de truncamiento es de orden h . Esto es, si f es de clase C^2 , el error de truncamiento decrece linealmente con h .

6.1.2. Fórmulas de 3 puntos

Se puede mejorar el orden del error de las aproximaciones anteriores, usando más puntos de evaluación. Para ello, se desarrolla el polinomio de Taylor de f en x_0 hasta el término cuadrático y evaluando dicho polinomio en $x = x_0 + h$ y $x = x_0 - h$

$$\begin{aligned} f(x_0 + h) &= f(x_0) + h f'(x_0) + \frac{h^2}{2} f''(x_0) + \frac{h^3}{6} f'''(\xi_1) \\ f(x_0 - h) &= f(x_0) - h f'(x_0) + \frac{h^2}{2} f''(x_0) - \frac{h^3}{6} f'''(\xi_2) \end{aligned}$$

para $\xi_1 \in [x_0, x_0 + h]$ y $\xi_2 \in [x_0 - h, x_0]$. Restando ambas expresiones, y despejando $f'(x_0)$:

$$f'(x_0) = \frac{f(x_0 + h) - f(x_0 - h)}{2h} + \frac{h^2}{6} f'''(\xi) \quad (6.5)$$

donde se ha considerado que existe $\xi \in [x_0 - h, x_0 + h]$ tal que $f'''(\xi)$ es igual al promedio $\frac{f'''(\xi_1) + f'''(\xi_2)}{2}$.

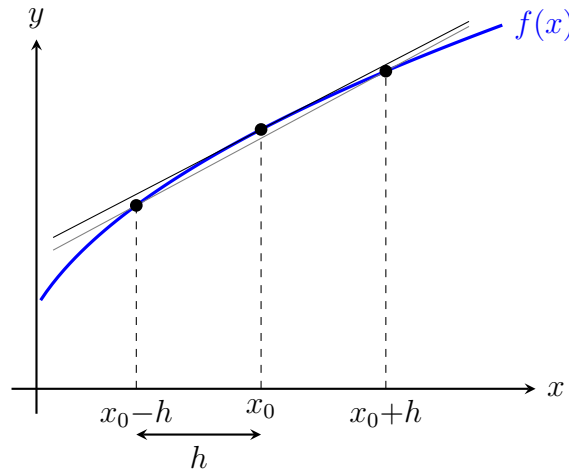


Figura 6.2: Diferencias finitas centrada de 3 puntos.

La ecuación(6.5) proporciona una fórmula centrada en x_0 denominada *regla de tres puntos centrada*, cuyo error de truncamiento es de orden h^2 . Es decir, en caso de que $f \in C^3$, no sólo se obtiene un mejor orden del error, si no que se sigue conservando la cantidad de evaluaciones de la función f como en las reglas de 2 puntos que vimos anteriormente.

De manera análoga, se pueden obtener fórmulas en diferencias finitas de 3 puntos adelantada y atrasada, ambas también con orden h^2 para el error (queda la deducción de las mismas a cargo del lector):

$$f'(x_0) = \frac{-3f(x_0) + 4f(x_0 + h) - f(x_0 + 2h)}{2h} + \frac{h^2}{3}f'''(\xi) \quad (6.6)$$

$$f'(x_0) = \frac{f(x_0 - 2h) - 4f(x_0 - h) + 3f(x_0)}{2h} + \frac{h^2}{3}f'''(\xi) \quad (6.7)$$

Se puede mejorar aún más el orden del error de truncamiento, agregando más puntos. A modo de ejemplo, se muestra la regla de 5 puntos centrada:

$$f'(x_0) = \frac{f(x_0 - 2h) - 8f(x_0 - h) + 8f(x_0 + h) - f(x_0 + 2h)}{12h} + \frac{h^4}{30}f^{(5)}(\xi) \quad (6.8)$$

Entonces, haciendo el desarrollo en series de Taylor en diferentes puntos, y realizando una combinación lineal adecuada de esos desarrollos, eliminando términos de derivadas más bajas. Se puede apreciar que a medida que agrego más puntos, se incrementa el costo computacional (mayor cantidad de evaluaciones de la función) y el orden del error de truncamiento exige que la función tenga mayor orden de derivación (por ejemplo, en la ecuación (6.8) f debe ser C^5).

6.1.3. Error de truncamiento vs. error de redondeo

En las fórmulas vistas anteriormente, se muestra el *error de truncamiento*, proveniente de *truncar* la serie de Taylor en los desarrollos propuestos. Se analizará ahora qué ocurre con el error de redondeo al aplicar la fórmula de diferencias finitas, y cómo afecta a la estimación de la derivada.

Como ejemplo, si se utiliza la fórmula de 3 puntos centrada y se designa con $\tilde{f}(x)$ a la representación en punto flotante de la evaluación de la función $f(x)$, entonces:

$$f(x_0 + h) = \tilde{f}(x_0 + h) + e_1$$

$$f(x_0 - h) = \tilde{f}(x_0 - h) + e_2$$

donde e_1 y e_2 son los errores de redondeo de dicha representación.

Luego, el error en la aproximación de la derivada con la fórmula centrada de 3 puntos nos queda:

$$f'(x_0) - \frac{\tilde{f}(x_0 + h) - \tilde{f}(x_0 - h)}{2h} = \frac{e_1 - e_2}{2h} - \frac{h^2}{6} f'''(\xi)$$

donde el primer término corresponde al error de redondeo y el segundo al error de truncamiento. Si se supone que e_1 y e_2 están acotados por algún valor $\epsilon > 0$, y que existe $M > 0$ que acota a la derivada tercera de f , entonces:

$$\left| f'(x_0) - \frac{\tilde{f}(x_0 + h) - \tilde{f}(x_0 - h)}{2h} \right| \leq \frac{\epsilon}{h} + \frac{h^2}{6} M \quad (6.9)$$

De la Ec. (6.9) se puede observar que si $f \in C^3$, para que el error de truncamiento $h^2 M/6$ sea pequeño, se debe reducir el valor de h , pero esto implica que ϵ/h crezca, es decir, que tome importancia el error de redondeo.

6.1.4. Extrapolación de Richardson

Hay un procedimiento muy original que permite reducir el error de truncamiento, utilizando una combinación de dos aproximaciones para diferentes valores de h , denominado *Extrapolación de Richardson*. En este curso no se profundizará en este concepto, pero se mostrará con un ejemplo el procedimiento a seguir. El lector puede ampliar sus conocimientos en la bibliografía recomendada.

Si se desarrolla el polinomio de Taylor centrado en x_0 para $x = x_0 + h$ y $x = x_0 - h$, se tiene:

$$f(x_0 + h) = f(x_0) + h f'(x_0) + \frac{h^2}{2} f''(x_0) + \frac{h^3}{3!} f'''(x_0) + \frac{h^4}{4!} f^{(4)}(x_0) + \dots \quad (6.10)$$

$$f(x_0 - h) = f(x_0) - h f'(x_0) + \frac{h^2}{2} f''(x_0) - \frac{h^3}{3!} f'''(x_0) + \frac{h^4}{4!} f^{(4)}(x_0) - \dots \quad (6.11)$$

Restando ambas expresiones y despejando $f'(x_0)$ obtenemos una aproximación centrada de 3 puntos:

$$f'(x_0) = \frac{f(x_0 + h) - f(x_0 - h)}{2h} - \left[\frac{h^2}{3!} f'''(x_0) + \frac{h^4}{5!} f^{(5)}(x_0) + \dots \right]$$

que la podemos expresar como:

$$f'(x) = \phi(h) - [a_2 h^2 + a_4 h^4 + a_6 h^6 + \dots] \quad (6.12)$$

donde se ha definido $\phi(h)$ a la estimación de la derivada (como función de h), y el corchete es el término de error de truncamiento.

El término $a_2 h^2$ es dominante sobre los demás, por lo que, si se quiere mejorar el error de truncamiento, sería conveniente eliminar ese término. Para ello, realizando el mismo procedimiento anterior para un paso $\frac{h}{2}$:

$$f'(x_0) = \frac{f(x_0 + \frac{h}{2}) - f(x_0 - \frac{h}{2})}{2 \cdot \frac{h}{2}} - \left[a_2 \frac{h^2}{4} + a_4 \frac{h^4}{16} + a_6 \frac{h^6}{64} + \dots \right]$$

que en términos del funcional ϕ ,

$$f'(x_0) = \phi\left(\frac{h}{2}\right) - \left[a_2 \frac{h^2}{4} + a_4 \frac{h^4}{16} + a_6 \frac{h^6}{64} + \dots \right] \quad (6.13)$$

Si se multiplica por 4 la Ec. (6.13) y se le resta la Ec.(6.12):

$$3f'(x_0) = 4 \phi\left(\frac{h}{2}\right) - \phi(h) - \frac{3}{4} a_4 h^4 - \frac{15}{16} a_6 h^6 - \dots$$

Por lo tanto:

$$f'(x_0) = \frac{4}{3} \phi\left(\frac{h}{2}\right) - \frac{1}{3} \phi(h) - \left[\frac{1}{4} a_4 h^4 + \frac{5}{16} a_6 h^6 + \dots \right]$$

Es decir que tomando como estimación de la derivada en x_0 :

$$f'(x_0) \approx \frac{4}{3} \phi\left(\frac{h}{2}\right) - \frac{1}{3} \phi(h)$$

donde $\phi(x)$ es la fórmula de diferencias finitas centrada de 3 puntos, el error de truncamiento está determinado por el término $a_4 h^4/4$. En conclusión, con una combinación lineal adecuada de dos aproximaciones de orden $\mathcal{O}(h^2)$ se puede obtener una nueva aproximación a la derivada cuyo error de truncamiento es de orden $\mathcal{O}(h^4)$.

Este proceso podría repetirse: si llamamos $\psi(h) = \frac{4}{3} \phi\left(\frac{h}{2}\right) - \frac{1}{3} \phi(h)$, podemos hacer una combinación lineal entre $\psi(h)$ y $\psi\left(\frac{h}{2}\right)$ para eliminar el término en h^4 , y así sucesivamente.

6.1.5. Derivadas de orden superior

También se pueden obtener fórmulas en diferencias finitas para derivadas de orden superior. A modo de ejemplo, se muestra un procedimiento similar a lo que se ha desarrollado en los casos anteriores.

Para ello, se consideran los desarrollos en series de Taylor (6.10) y (6.11), y se suman se obtiene:

$$f(x_0 - h) + f(x_0 + h) = 2f(x_0) + h^2 f''(x_0) + \left[\frac{h^4}{12} f^{(4)}(x_0) + \dots \right]$$

donde se puede apreciar que los términos de derivadas impares se cancelan. Despejando $f''(x_0)$, y considerando que $f \in C^4$, se puede demostrar que:

$$f''(x_0) = \frac{f(x_0 + h) - 2f(x_0) + f(x_0 - h)}{h^2} - \frac{h^2}{12} f^{(4)}(\xi) \quad (6.14)$$

para algún $\xi \in [x_0 - h, x_0 + h]$.

Análogamente, involucrando 5 puntos se puede obtener una fórmula en diferencias finitas para la derivada tercera, también de orden $\mathcal{O}(h^2)$:

$$f'''(x) \approx \frac{f(x + 2h) - 2f(x + h) + 2f(x - h) - f(x - 2h)}{2h^3}$$

¿Se anima a deducir esta fórmula, y determinar hasta qué derivada continua de f se necesita para garantizar el orden del error de truncamiento mencionado?

6.1.6. Derivadas a partir de polinomios interpolantes

Para las fórmulas en diferencias finitas desarrolladas, se han considerado puntos igualmente espaciados, según el parámetro h . Esto se puede generalizar para puntos que no están igualmente espaciados, considerando polinomios interpoladores.

En este sentido, consideremos n puntos $x_1, x_2, \dots, x_n$, y el polinomio interpolante a la función f en esos puntos en la forma de Lagrange y su respectivo término de error:

$$f(x) = \sum_{i=1}^n f(x_i) L_{n-1,i}(x) + \frac{1}{n!} f^{(n)}(\xi_x) \prod_{j=1}^n (x - x_j)$$

con

$$L_{n-1,i}(x) = \prod_{\substack{j=1; \\ j \neq i}}^n \frac{(x - x_j)}{(x_i - x_j)}$$

Si derivamos $f(x)$ se obtiene:

$$f'(x) = \sum_{i=1}^n f(x_i) L'_{n-1,i}(x) + \frac{1}{n!} \frac{d}{dx} (f^{(n)}(\xi_x)) \prod_{j=1}^n (x - x_j) + \frac{f^{(n)}(\xi_x)}{n!} \frac{d}{dx} \left(\prod_{j=1}^n (x - x_j) \right)$$

El interés es estimar la derivada en los puntos de referencia x_k que son los datos. Esto genera que el segundo término de la expresión anterior se anule, obteniendo la forma general para la derivada en los nodos:

$$f'(x_k) = \sum_{i=1}^n f(x_i) L'_{n-1,i}(x_k) + \frac{f^{(n)}(\xi_{x_k})}{n!} \prod_{\substack{j=1; \\ j \neq k}}^n (x_k - x_j) \quad (6.15)$$

Observación 20. Si se consideran en particular 3 puntos igualmente espaciados, se obtienen las fórmulas en diferencias finitas que se estudiaron anteriormente.

6.2. Integración numérica o cuadratura

El objetivo de esta sección es hallar fórmulas que permitan calcular integrales (aproximadas) pero de cualquier función. Cuando decimos *aproximadas* queremos decir con tanta precisión como se desee.

Queremos calcular por ejemplo

$$\int_0^A e^{-x^2} dx \quad \text{o} \quad \int_6^5 \frac{t^3}{e^t - 1} dt$$

que no tienen una expresión analítica.

Integrales difíciles de resolver analíticamente suelen aparecer al calcular el área de superficies de revolución. Recordamos que el área de la superficie de revolución que se obtiene al girar el gráfico de $y = f(x)$ alrededor del eje x , para $a \leq x \leq b$ es

$$\int_a^b 2\pi f(x) \sqrt{1 + f'(x)^2} dx.$$

Rara vez el integrando de esta expresión se podrá integrar analíticamente, y son muy útiles los métodos numéricos. Estos no dan una fórmula cerrada para el resultado, pero permiten calcular la integral con tanta precisión como se desee.

6.2.1. Fórmulas de Newton-Côtes

Supongamos que se desea calcular la integral

$$\int_a^b f(x) dx$$

para una función dada f .

La idea de las fórmulas de Newton-Côtes es interpolar la función f en n puntos equiespaciados con un polinomio de grado $\leq n - 1$ e integrar el polinomio.

Por ejemplo. Para $n = 1$ tomamos el punto medio, y el polinomio de grado cero que interpola en ese punto es la función constante $p_1(x) = f(\frac{a+b}{2})$. Resulta

$$\int_a^b f(x) dx \approx \int_a^b p_1(x) dx = \int_a^b f\left(\frac{a+b}{2}\right) dx = \underbrace{(b-a)f\left(\frac{a+b}{2}\right)}_{Q_1(f;a,b)}.$$

Esta fórmula se conoce como *Regla del Rectángulo*, porque estamos calculando el área del rectángulo con base $(b-a)$ y altura $f(\frac{a+b}{2})$.

Para $n = 2$ podemos escribir el polinomio p_2 que interpola a f en $x = a$ y en $x = b$ de la siguiente manera

$$p_2(x) = \frac{b-x}{b-a} f(a) + \frac{x-a}{b-a} f(b).$$

Luego

$$\int_a^b f(x) dx \approx \int_a^b p_2(x) dx = \cdots = \underbrace{(b-a) \left(\frac{1}{2} f(a) + \frac{1}{2} f(b) \right)}_{Q_2(f;a,b)}.$$

Esta fórmula se conoce como *Regla del Trapecio*, porque estamos calculando el área del trapecio con bases $f(a)$, $f(b)$ y altura $(b - a)$.

Para $n = 3$ podemos escribir el polinomio p_3 que interpola a f en $x = a$, $x = m = \frac{a+b}{2}$ y en $x = b$ de la siguiente manera

$$p_3(x) = \frac{(b-x)(m-x)}{(b-a)(m-a)}f(a) + \frac{(b-x)(a-x)}{(b-m)(a-m)}f(m) + \frac{(x-a)(x-m)}{(b-a)(b-m)}f(b).$$

Luego

$$\int_a^b f(x) dx \approx \int_a^b p_3(x) dx = \cdots = \underbrace{(b-a) \left(\frac{1}{6}f(a) + \frac{4}{6}f(m) + \frac{1}{6}f(b) \right)}_{Q_3(f;a,b)}.$$

Esta fórmula se conoce como *Regla de Simpson*.

Las operaciones que no escribimos y que irían en $\cdots$ son muy laboriosas. Lo importante de entender aquí son dos cosas:

- (1) Las fórmulas de integración numérica consisten en evaluar f en distintos puntos del intervalo, multiplicar esos valores por otros números, sumar, y finalmente multiplicar por la longitud del intervalo.
- (2) Las fórmulas deberían ser exactas, es decir

$$\int_a^b f(x) dx = Q_n(f, a, b)$$

cuando f es un polinomio de grado $\leq n - 1$. ¿Por qué? Porque si interpolamos un polinomio de grado $\leq n - 1$ en n puntos, obtenemos el mismo polinomio original. Entonces integrar ese polinomio nos da la integral de la función de partida.

Definición 6.1. Una fórmula de integración numérica o de cuadratura consiste de n puntos $x_1, x_2, \dots, x_n$ en el intervalo y n pesos $w_1, w_2, \dots, w_n$ de modo que

$$\int_a^b f(x) dx \approx (b-a) \sum_{i=1}^n w_i f(x_i) =: Q_n(f; a, b).$$

Para hallar los w_i correspondientes a la fórmula de Newton-Côtes de n puntos, consideramos el intervalo $[0, 1]$ y recordamos que debería cumplirse que

$$\int_a^b f(x) dx = Q_n(f, a, b) = \sum_{i=1}^n w_i f(x_i)$$

para todo polinomio de grado $\leq n - 1$. En particular debería cumplirse para $f(x) = 1$,

para $f(x) = x$, para $f(x) = x^2, \dots$, y para $f(x) = x^{n-1}$. Luego

$$\begin{array}{ll} f(x) = 1 & \int_0^1 f(x) dx = 1 = w_1 + w_2 + \dots + w_n \\ f(x) = x & \int_0^1 x dx = \frac{1}{2} = w_1 x_1 + w_2 x_2 + \dots + w_n x_n \\ f(x) = x^2 & \int_0^1 x^2 dx = \frac{1}{3} = w_1 x_1^2 + w_2 x_2^2 + \dots + w_n x_n^2 \\ \vdots & \\ f(x) = x^{n-1} & \int_0^1 x^{n-1} dx = \frac{1}{n} = w_1 x_1^{n-1} + w_2 x_2^{n-1} + \dots + w_n x_n^{n-1}. \end{array}$$

Recordemos que los puntos $x_1, \dots, x_n$ son equiespaciados en $[0, 1]$ es decir, $x_1 = 0$, $x_2 = 1/(n-1)$, $x_3 = 2/(n-1)$..., $x_n = 1$ (`x=linspace(0,1,n)`) y que las incógnitas son los pesos w_i de la fórmula. Entonces vemos que los pesos w_i son las soluciones del siguiente sistema de ecuaciones:

$$\begin{array}{l} w_1 + w_2 + \dots + w_n = 1 \\ w_1 x_1 + w_2 x_2 + \dots + w_n x_n = \frac{1}{2} \\ w_1 x_1^2 + w_2 x_2^2 + \dots + w_n x_n^2 = \frac{1}{3} \\ \vdots \\ w_1 x_1^{n-1} + w_2 x_2^{n-1} + \dots + w_n x_n^{n-1} = \frac{1}{n}. \end{array}$$

En forma matricial

$$\begin{bmatrix} 1 & 1 & \dots & 1 \\ x_1 & x_2 & \dots & x_n \\ x_1^2 & x_2^2 & \dots & x_n^2 \\ \vdots & \vdots & \ddots & \vdots \\ x_1^{n-1} & x_2^{n-1} & \dots & x_n^{n-1} \end{bmatrix} \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_n \end{bmatrix} = \begin{bmatrix} 1 \\ \frac{1}{2} \\ \vdots \\ \frac{1}{n} \end{bmatrix}$$

Por lo tanto, los pesos de la fórmula de Newton-Côtes se encuentran resolviendo ese sistema. Hay otras maneras de hallar los pesos, que son más precisas si n es grande, pero para $n \leq 10$ (que es lo que utilizaremos nosotros), esta manera es práctica.

El siguiente código calcula las abscisas x_i y los pesos w_i para el intervalo $[0, 1]$ y un entero positivo n .

```
function w = pesosNC(n)
% function w = pesosNC(n)
% se calculan los pesos
% de la formula de Newton-Cotes de n puntos

x = linspace(0,1,n);
A = ones(n,n);
```

```

for i=2:n
    A(i,:) = A(i-1,:) .* x;
end
b = 1./(1:n)';

w = A\b;

```

El siguiente código calcula la integral aproximada de una función f utilizando la fórmula de Newton-Côtes de n puntos. Las abscisas se calculan tomando n puntos equiespaciados en el intervalo $[a, b]$ (usando `linspace`).

```

function Q = integralNC(f,a,b,n)
% function Q = integralNC(f,a,b,n)
% se calcula la integral de f sobre [a,b]
% aproximadamente, usando la formula
% de Newton-Cotes de n puntos

w = pesosNC(n);

x = linspace(a,b,n);
fx = f(x);

Q = (b-a)*(fx*w);

```

Análisis del error para las fórmulas de Newton-Côtes

Dada una función suave f en el intervalo $[a, b]$, ¿qué error se comete al calcular la integral con la fórmula de Newton-Cotes de n puntos?

Para saber esto, recordemos que

$$Q_n(f; a, b) = \int_a^b p_n(x) dx$$

con p_n el polinomio de grado $\leq n-1$ que interpola a f en n puntos. Este polinomio satisface, según el Corolario 5.4

$$\max_{x \in [a, b]} |f(x) - p_n(x)| \leq \frac{M_n}{n} \frac{|b-a|^n}{(n-1)^n},$$

con $M_n = \max_{\xi \in [a, b]} |f^{(n)}(\xi)|$.

Por lo tanto

$$\begin{aligned}
 \left| \int_a^b f(x) dx - Q_n(f; a, b) \right| &= \left| \int_a^b f(x) dx - \int_a^b p_n(x) dx \right| \\
 &= \left| \int_a^b f(x) - p_n(x) dx \right| \leq \int_a^b |f(x) - p_n(x)| dx \\
 &\leq |b-a| \frac{M_n}{n} \frac{|b-a|^n}{(n-1)^n} = \frac{M_n}{n} \frac{|b-a|^{n+1}}{(n-1)^n}
 \end{aligned}$$

Hemos demostrado entonces el siguiente teorema:

Teorema 6.1. Si f es una función C^n en $[a, b]$, entonces

$$\left| \int_a^b f(x) dx - Q_n(f; a, b) \right| \leq \frac{M_n |b - a|^{n+1}}{n(n-1)^n},$$

con $M_n = \max_{\xi \in [a, b]} |f^n(\xi)|$.

Esta cota del error entre la integral exacta y la aproximada tiene el mismo problema que la cota del error de interpolación: Hay muchas funciones para las que ese término **no** tiende a cero (cuando $M_n \rightarrow \infty$ rápidamente.) Este problema se resuelve utilizando las fórmulas de Newton-Côtes compuestas.

6.2.2. Fórmulas de Newton-Côtes compuestas

La idea de estas fórmulas es integrar el interpolante polinomial a trozos:

Particionamos el intervalo $[a, b]$ en L sub-intervalos de longitud $h = \frac{b-a}{L}$. Definimos

$$y_1 = a, \quad y_2 = a+h, \quad y_3 = a+2h, \quad \dots \quad y_L = a+(L-1)h = b-h, \quad y_{L+1} = a+Lh = b,$$

y observamos que

$$\int_a^b f(x) dx = \int_{y_1}^{y_2} f(x) dx + \int_{y_2}^{y_3} f(x) dx + \dots + \int_{y_L}^{y_{L+1}} f(x) dx = \sum_{i=1}^L \int_{y_i}^{y_{i+1}} f(x) dx.$$

Luego, aproximamos cada integral con una fórmula de Newton-Côtes con n fijo:

$$\int_{y_i}^{y_{i+1}} f(x) dx \approx Q_n(f, y_i, y_{i+1})$$

y resulta

$$\int_a^b f(x) dx \approx \underbrace{\sum_{i=1}^L Q_n(f, y_i, y_{i+1})}_{Q_n^L(f; a, b)}.$$

Para calcular una integral con este método, podemos usar el siguiente código.

```
function Q = NCcompuesta(f,a,b,L,n)
% function Q = NCcompuesta(f,a,b,L,n)
% aproxima la integral de f sobre [a,b]
% utilizando la formula de Newton-Cotes compuesta
% de n puntos, subdividiendo en L subintervalos

y = linspace(a,b,L+1);

Q = 0;

for i=1:L
    Q = Q + integralNC(f,y(i),y(i+1),n);
end
```

Esta implementación es muy ineficiente, pues cada vez que se llama a `integralNC` se calculan los pesos de Newton-Côtes, y estos son los mismos en todos los sub-intervalos. La siguiente, es una implementación un poco más eficiente.

```
function Q = intNCcompuesta(f,a,b,L,n)
% function Q = intNCcompuesta(f,a,b,L,n)
% aproxima la integral de f sobre [a,b]
% utilizando la formula de Newton-Cotes compuesta
% de n puntos, subdividiendo en L subintervalos

y = linspace(a,b,L+1);
h = (b-a)/L;

% calculamos los pesos una sola vez
w = pesosNC(n);

Q = 0;
for i=1:L
    x = linspace(y(i),y(i+1),n);
    fx = f(x);
    Q = Q + h*(fx*w);
end
```

Observación 21. La fórmula compuesta correspondiente a $n = 2$ se llama *fórmula del trapecio compuesta* y la correspondiente a $n = 3$ se llama *fórmula de Simpson compuesta*.

Análisis del error para las fórmulas de Newton-Côtes compuestas

Observemos lo siguiente:

$$\begin{aligned} \left| \int_a^b f(x) dx - Q_n^L(f, a, b) \right| &= \left| \sum_{i=1}^L \int_{y_i}^{y_{i+1}} f(x) dx - \sum_{i=1}^L Q_n(f, y_i, y_{i+1}) \right| \\ &\leq \sum_{i=1}^L \left| \int_{y_i}^{y_{i+1}} f(x) dx - Q_n(f, y_i, y_{i+1}) \right| \\ &\leq \sum_{i=1}^L \frac{M_n}{n} \frac{\left(\frac{|b-a|}{L}\right)^{n+1}}{(n-1)^n} = L \frac{M_n}{n} \frac{\left(\frac{|b-a|}{L}\right)^{n+1}}{(n-1)^n} = \frac{M_n}{n} \frac{|b-a|^{n+1}}{(n-1)^n} \frac{1}{L^n}. \end{aligned}$$

Aquí hemos usado, en cada sub-intervalo $[y_i, y_{i+1}]$ la estimación del error dada por el Teorema 6.1. Hemos entonces demostrado el siguiente teorema.

Teorema 6.2. Si f es una función C^n en $[a, b]$, entonces

$$\left| \int_a^b f(x) dx - Q_n^L(f; a, b) \right| \leq \frac{M_n |b-a|^{n+1}}{n(n-1)^n} \frac{1}{L^n}, \quad (6.16)$$

con $M_n = \max_{\xi \in [a,b]} |f^n(\xi)|$.

Observación 22. Observemos que si $f \in C^n$, entonces M_n es un número finito, y el lado derecho de la fórmula anterior tiende a cero cuando $L \rightarrow \infty$. Es decir, el error tiende a cero a medida que tomamos L cada vez más grande. Además, cuando mayor sea n , más rápido tenderá a cero el error.

Pero ¡cuidado! El error no tiende a cero si dejamos L fijo y hacemos tender n a infinito. ¿Por qué? Pensar.

6.2.3. Cuadratura de Gauss

Las fórmulas de Newton-Cotes se obtuvieron integrando polinomios interpolantes con los nodos igualmente espaciados el intervalo de integración. En este sentido, si se tienen n puntos, se integra un polinomio de grado $n - 1$, consiguiendo que esta fórmula sea exacta para funciones polinómicas de grado menor o igual a $n - 1$.

La pregunta que surge es si se podrán elegir los puntos de manera diferente, de modo tal que podamos mejorar el grado de precisión, es decir, con n puntos poder integrar de manera exacta polinomios de grado mayor a $n - 1$.

Para ilustrar este interrogante, en la siguiente gráfica se muestra la regla del trapecio (2 puntos) aplicada en un intervalo $[a, b]$ y una selección de dos puntos diferentes (internos al intervalo) para generar otro polinomio lineal. Claramente se puede apreciar que la segunda opción mejora la aproximación de la integral de la función en el intervalo dado.

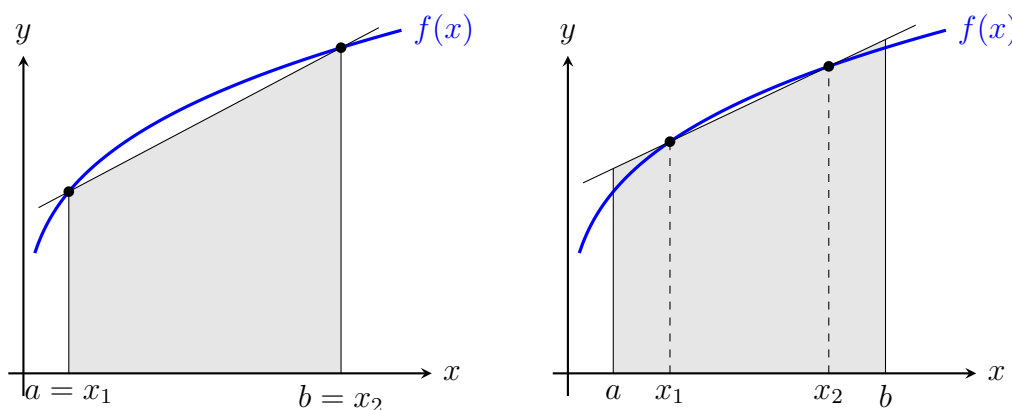


Figura 6.3: A la izquierda se muestra el polinomio lineal usando los extremos (regla de trapecio). A la derecha el polinomio lineal usando dos nodos internos.

En esta parte se describirá un proceso particular para hallar esos puntos (y sus respectivos pesos) para que la aproximación por cuadratura

$$\int_a^b f(x) dx \approx \sum_{i=1}^n c_i f(x_i)$$

minimice el error esperado. Este nuevo proceso de cuadratura se denomina *Cuadratura de Gauss*.

Entonces, si quiero encontrar los n nodos $x_1, x_2, \dots, x_n$ y los respectivos coeficientes $c_1, c_2, \dots, c_n$, hay $2n$ parámetros por determinar. Es razonable esperar que la fórmula obtenida sea exacta para polinomios de grado menor o igual a $2n - 1$.

Se ilustra para el caso $n = 2$ un procedimiento para hallar los nodos x_1, x_2 en el intervalo $[-1, 1]$ y los pesos c_1, c_2 , para entender el concepto, y luego se generalizará.

Si se quiere que la fórmula de cuadratura

$$\int_{-1}^1 f(x) dx \approx c_1 f(x_1) + c_2 f(x_2)$$

sea exacta para polinomios de grado menor o igual a $2n - 1 = 2 \cdot 2 - 1 = 3$. Entonces, debería cumplirse la igualdad en particular para $f = 1$, $f = x$, $f = x^2$ y $f = x^3$. Esto es:

$$\int_{-1}^1 1 dx = 2 = c_1 + c_2 \quad (6.17)$$

$$\int_{-1}^1 x dx = 0 = c_1 x_1 + c_2 x_2 \quad (6.18)$$

$$\int_{-1}^1 x^2 dx = \frac{2}{3} = c_1 x_1^2 + c_2 x_2^2 \quad (6.19)$$

$$\int_{-1}^1 x^3 dx = 0 = c_1 x_1^3 + c_2 x_2^3 \quad (6.20)$$

$$(6.21)$$

Si se resuelve este sistema (no lo haremos aquí), la solución es:

$$c_1 = 1, \quad c_2 = 1, \quad x_1 = -\frac{\sqrt{3}}{3}, \quad x_2 = \frac{\sqrt{3}}{3}$$

Polinomios de Legendre

Este procedimiento se podría hacer para mayor cantidad de nodos, pero se describirá otro método más general, pero no tal intuitivo como el anterior. Para ello, se introduce el concepto de *Polinomios Ortogonales*, en el sentido que la integral del producto de 2 de ellos es 0. Para nuestro problema, se considera un conjunto particular de estos polinomios: los *Polinomios de Legendre*, $\{P_0, P_1, \dots, P_n, \dots\}$ que cumplen estas propiedades:

(1) Para cada n , P_n es un polinomio monómico¹ de grado n .

(2) $\int_{-1}^1 P(x)P_n(x) dx = 0$ siempre que $P(n)$ sea cualquier polinomio de grado menor a n .

Las raíces de estos polinomios son distintas, se ubican en el intervalo $[-1, 1]$ y son justamente los nodos que necesitamos, y a partir de ellos, se pueden determinar los pesos correspondientes para la regla de cuadratura. Esto se establece con el siguiente resultado:

¹Los polinomios monómicos son los que tienen coeficiente 1 en el término principal

Teorema 6.3. Si $x_1, x_2, \dots, x_n$ son las raíces del polinomio de Legendre $P_n(x)$, y se define:

$$c_k = \int_{-1}^1 \prod_{\substack{j=1; \\ j \neq k}}^n \frac{x - x_j}{x_k - x_j} dx, \quad \text{para } k = 1, 2, \dots, n$$

Entonces:

$$\int_{-1}^1 P(x) dx = \sum_{k=1}^n c_k P(x_k)$$

para cualquier polinomio $P(x)$ de grado menor a $2n$.

La demostración de este teorema no es complicada y se deja para el lector².

En la bibliografía se puede encontrar tablas con los polinomios de Legendre, sus raíces en el intervalo $[-1, 1]$ y los pesos correspondientes para la cuadratura, como se resume para algunos casos en la tabla 6.1.

n	Polinomio $P_n(x)$	Raíces x_i	Pesos w_i
0	$P_0(x) = 1$	No se usa en cuadratura	
1	$P_1(x) = x$	$x_1 = 0$	$w_1 = 2$
2	$P_2(x) = x^2 - \frac{1}{3}$	$x_1 = -1/\sqrt{3} \approx -0.577350$ $x_2 = +1/\sqrt{3} \approx +0.577350$	$w_1 = 1$ $w_2 = 1$
3	$P_3(x) = x^3 - \frac{3}{5}x$	$x_1 = -\sqrt{3/5} \approx -0.774597$ $x_2 = 0$ $x_3 = +\sqrt{3/5} \approx +0.774597$	$w_1 = \frac{5}{9} \approx 0.55$ $w_2 = \frac{8}{9} \approx 0.88$ $w_3 = \frac{5}{9} \approx 0.55$
4	$P_4(x) = x^4 - \frac{6}{7}x^2 + \frac{3}{35}$	$x_1 \approx -0.861136$ $x_2 \approx -0.339981$ $x_3 \approx +0.339981$ $x_4 \approx +0.861136$	$w_1 \approx 0.347855$ $w_2 \approx 0.652145$ $w_3 \approx 0.652145$ $w_4 \approx 0.347855$

Tabla 6.1: Polinomios de Legendre, raíces y pesos para cuadratura de Gauss-Legendre

Código en Octave

En este curso, se proponen códigos en el mismo sentido que se hizo con la cuadratura de Newton-Cotes.

El primero de ellos, calcula los pesos y nodos en el intervalo $[-1, 1]$, recibiendo n el número de nodos.

²Puede verla en Burden, R. L. y Faires, J. D. *Análisis numérico*, 7ma edición. International Thomson Editores, 2002.

```

function [x, w] = gauss_xw(n)
% [x, w] = gauss_xw(n)
% Genera las abscisas y pesos para la Cuadratura Gauss-Legendre.
% n: numero de puntos de integracion
% x: abscisas de la cuadratura
% w: pesos de la cuadratura
x = zeros(n,1);
w = x;
m = (n+1)/2;
for ii=1:m
    z = cos(pi*(ii-.25)/(n+.5));           % estimado Inicial.
    z1 = z+1;
    while abs(z-z1)>eps
        p1 = 1;
        p2 = 0;
        for jj = 1:n
            p3 = p2;
            p2 = p1;
            p1 = ((2*jj-1)*z*p2-(jj-1)*p3)/jj; % El polinomial. Legendre
        endfor
        pp = n*(z*p1-p2)/(z^2-1);           % La L.P. Derivada.
        z1 = z;
        z = z1-p1/pp;
    endwhile
    x(ii) = -z;                             % Construye las abscisas.
    x(n+1-ii) = z;
    w(ii) = 2/((1-z^2)*(pp^2));             % Construye los pesos.
    w(n+1-ii) = w(ii);
endfor

```

Esto permite estimar la integral de una función f en el intervalo $[-1, 1]$, pero se puede extender este cálculo a cualquier intervalo $[a, b]$.

Cuadratura de Gauss en intervalo arbitrario

Para aplicar la fórmula en un intervalo arbitrario $[a, b]$ se utiliza el cambio de variable lineal

$$x = \frac{b-a}{2}t + \frac{a+b}{2}, \quad dx = \frac{b-a}{2}dt,$$

que transforma $t \in [-1, 1]$ en $x \in [a, b]$. La integral queda entonces

$$\int_a^b f(x) dx = \frac{b-a}{2} \int_{-1}^1 f\left(\frac{b-a}{2}t + \frac{a+b}{2}\right) dt \approx \frac{b-a}{2} \sum_{i=1}^n w_i f\left(\frac{b-a}{2}x_i + \frac{a+b}{2}\right),$$

donde x_i y w_i son las raíces y pesos de la cuadratura de Gauss en el intervalo $[-1, 1]$.

Implementando este cambio en Octave, el código nos queda:

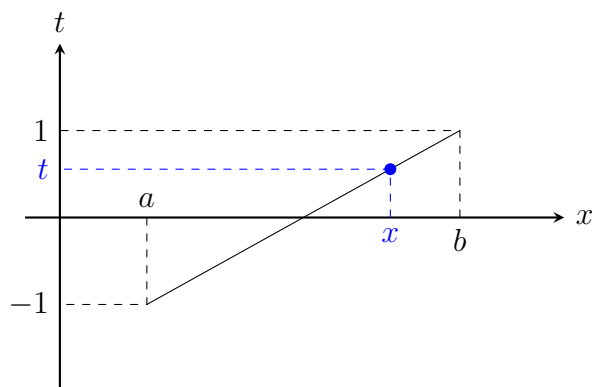


Figura 6.4: Transformación del intervalo $[a, b]$ en el intervalo $[-1, 1]$

```
function Q=cuad_gauss(f,a,b,n)
[x,w]=gauss_xw(n);

t=(b-a)/2*(x+1)+a;
Q=(b-a)/2*(w'*f(t));
```

Pero, en el mismo sentido que con la cuadratura de Newton-Cotes, interesa usar un pequeño número de puntos de evaluación n , se propone el siguiente código que calcula *cuadratura compuesta de Gauss*.

```
function Q=cuad_gauss_c(f,a,b,L,n)

[xg,w]=gauss_xw(n);

x=linspace(a,b,L+1);
h=(b-a)/L;
Q=0;
for i=1:L
    t=h/2*(xg+1)+x(i);
    Q+=h/2*(w'*f(t));
endfor
```

6.3. Consideraciones prácticas

Esta sección tiene como objetivo estudiar una metodología práctica para determinar de manera precisa la estimación de una integral definida. Es de especial interés prestar atención para relacionarlo con los ejercicios que se deberán resolver en la práctica del curso.

6.3.1. Cómo calcular integrales de funciones complicadas

Consideremos el caso en que queremos calcular la siguiente integral

$$\int_0^4 \sqrt{1 + (1 - \sin(x))^2} dx,$$

con un error menor a 10^{-12} . Podríamos comenzar a derivar la función para determinar, dado un n , cuál es el L necesario, en base a la cota (6.16) para tener un error menor que 10^{-12} , pero esto no es muy práctico. En lugar de eso, procedemos como sigue:

- (1) Definimos la función a integrar

```
>> g = @(x) sqrt(1+(1-sin(x)).^2)
```

- (2) Elegimos un n bajo, (probamos primero con 2 o 3) y un L no muy grande, por ejemplo 10

```
>> n = 2; L = 10;
```

- (3) Calculamos la integral con el método compuesto:

```
>> intNCcompuesta(g, 0, 4, L, n)
ans = 4.95947743348974
```

- (4) A partir de ahí vamos duplicando L y volviendo a calcular hasta que veamos que se tiene repetición de los dígitos que queremos tener:

```
>> L = 2*L; intNCcompuesta(g, 0, 4, L, n)
ans = 4.94665734614109
>> L = 2*L; intNCcompuesta(g, 0, 4, L, n)
ans = 4.94346518493875
>> L = 2*L; intNCcompuesta(g, 0, 4, L, n)
ans = 4.94266793995222
>> L = 2*L; intNCcompuesta(g, 0, 4, L, n)
ans = 4.94246867828007
>> L = 2*L; intNCcompuesta(g, 0, 4, L, n)
ans = 4.94241886595837
```

- (5) Si se ve que tarda mucho en converger, entonces aumentamos n y comenzamos de nuevo con L bajo:

```
>> n = 3; L = 10;
>> intNCcompuesta(g, 0, 4, L, n)
ans = 4.94238398369153
>> L = 2*L; intNCcompuesta(g, 0, 4, L, n)
ans = 4.94240113120463
```

```

>> L = 2*L; intNCcompuesta(g, 0, 4, L, n)
ans = 4.94240219162338
>> L = 2*L; intNCcompuesta(g, 0, 4, L, n)
ans = 4.94240225772269
>> L = 2*L; intNCcompuesta(g, 0, 4, L, n)
ans = 4.94240226185113
>> L = 2*L; intNCcompuesta(g, 0, 4, L, n)
ans = 4.94240226210912
>> L = 2*L; intNCcompuesta(g, 0, 4, L, n)
ans = 4.94240226212524
>> L = 2*L; intNCcompuesta(g, 0, 4, L, n)
ans = 4.94240226212626
>> L = 2*L; intNCcompuesta(g, 0, 4, L, n)
ans = 4.94240226212631
>> L = 2*L; intNCcompuesta(g, 0, 4, L, n)
ans = 4.94240226212630
>> L = 2*L; intNCcompuesta(g, 0, 4, L, n)
ans = 4.94240226212631
>> L = 2*L; intNCcompuesta(g, 0, 4, L, n)
ans = 4.94240226212631

```

Esta manera de calcular integrales nos ayuda a comprender que el método de integración compuesta converge cuando dejamos n fijo y hacemos crecer L , y también que converge más rápido si n es más grande (con mayor orden).

Observación 23. Otra cosa importante en este método es que L tiene que ir duplicándose, y no incrementando de a uno. Si se incrementa de a uno, entonces parece que converge porque la solución cambia poco, pero aún se está lejos de la precisión deseada.

Observación 24. Este procedimiento que se muestra para la cuadratura de Newton-Cotes compuesta, también es válida para la cuadratura de Gauss-Legendre. Simplemente se mostró aquí como ejemplo para una de ellas.

6.3.2. Estimación de integrales usando un conjunto de datos

Los métodos de integración compuesta que hemos visto hasta ahora nos sirven para estimar la integral de una función dada. Para ello, hemos usado ciertos puntos del intervalo, en los cuales hemos evaluado la función para aplicar la regla de cuadratura.

Supongamos ahora que no tenemos una expresión de la función, pero en su lugar tenemos puntos que la representan. En este caso, podemos aplicar las reglas de cuadratura utilizando esos datos.

Más precisamente, dados los puntos

$$(x_1, y_1) \quad (x_2, y_2) \quad \dots \quad (x_n, y_n)$$

(supongamos igualmente espaciados en sus abscisas, por simplicidad) utilizaremos los datos y_i como si fueran los $f(x_i)$ de alguna f en las reglas de cuadratura, es decir:

$$\int_{x_a}^{x_b} f(x) dx \approx h \sum_{i=1}^n w_i y_{a+i-1}$$

con n chico. Hemos llamado x_a y x_b los límites del subintervalo a usar, que involucran a los n puntos de integración, incluyéndolos a ambos extremos. Por ejemplo, si usamos una regla de simpson ($n = 3$), y queremos estimar la integral del subintervalo delimitado por x_3 y x_5 , tendremos:

$$\int_{x_3}^{x_5} f(x) dx \approx h(w_1y_3 + w_2y_4 + w_3y_5)$$

A continuación, mostramos los algoritmos en Octave que implementan la regla de trapecio y la regla de simpson.

```
function Q = trapcomp(x,y)
%
% function Q = trapcomp(x,y)
%
% Regla del trapecio compuesta para aproximar
% la integral de una funcion f en el intervalo
% [x(1), x(end)]. Se supone que y(i) = f(x(i)).

% Calculamos la cantidad L de subintervalos
% que determinan los datos
L = numel(x) - 1;

% Calculamos un vector que tiene la longitud
% de cada uno de los L subintervalos
deltax = diff(x); % deltax(i) = x(i+1)-x(i)

% Ahora aplicamos la regla del trapecio en
% cada intervalo [x(i),x(i+1)], para i = 1,2,...,L
Q = 0;
for i = 1:L
    Q = Q + .5*deltax(i)*(y(i)+y(i+1));
end
```

Podemos observar que en este caso, al tomar de a dos puntos consecutivos, calcula el h de separación para cada caso. Es decir, se puede usar el algoritmo para puntos no necesariamente equiespaciados. En el caso del próximo algoritmo (como se aclara en los comentarios del mismo) los puntos si se consideran igualmente distribuidos.

```
function Q = simpsoncomp(x,y)
%
% function Q = simpsoncomp(x,y)
%
% Regla de Simpson compuesta para aproximar
% la integral de una funcion f en el intervalo
% [x(1), x(end)]. Se supone que y(i) = f(x(i)),
% y que los puntos x(i) estan equidistribuidos
% y que son una catidad impar
```

```

% Calculamos la cantidad L de subintervalos
% que determinan los datos
L = numel(x) - 1;

% Chequeamos que haya una cantidad par de
% subintervalos
if rem(L,2)
    % Chequear en la linea de comandos
    % que devuelve rem(L,2) para distintos valores de L
    disp('Atencion: Tiene que dar una cantidad impar de datos')
    Q = NaN;
    return
end

% Para aplicar la regla de Simpson compuesta
% en L/2 intervalos consecutivos, necesitamos la longitud
% de cada intervalo.
h = (x(end)-x(1))/(L/2);

% Ahora aplicamos la regla de Simpson compuesta.
% Para entender la siguiente linea primero hay que
% escribir en papel como quedaria la formula.
Q = h/6 * (y(1) + 4*sum(y(2:2:end-1)) + 2*sum(y(3:2:end-2)) + y(end));

```

Observación 25. Como los nodos en este caso están dados, no podemos “inventar” datos, por lo que no se puede aplicar de forma directa una cuadratura de Gauss. ¿Por qué?